

Chapitre 2 : Les tris

12/09/2025

1 Introduction

Un algorithme de tri est, en informatique ou en mathématiques, un algorithme qui permet d'organiser une collection d'objets selon un ordre déterminé. Les objets à trier font donc partie d'un ensemble muni d'une relation d'ordre. Les ordres les plus utilisés sont l'ordre numérique et l'ordre lexicographique (dictionnaire). Suivant la relation d'ordre considérée, une même collection d'objet peut donner lieu à divers arrangements, pourtant il est possible de définir un algorithme de tri indépendamment de la fonction d'ordre utilisée. Celui-ci ne fera qu'utiliser une certaine fonction d'ordre correspondant à une relation d'ordre qui doit permettre de comparer tout couple d'éléments de la collection.

2 Classification

La classification des algorithmes de tri est très importante, car elle permet de choisir l'algorithme le plus adapté au problème traité, tout en tenant compte des contraintes imposées par celui-ci.

On distingue, tout d'abord, les algorithmes de tri d'application générale, procédant par comparaisons entre des paires d'éléments, et les algorithmes plus spécialisés faisant des hypothèses restrictives sur la structure des données entrées (par exemple : le tri par comptage, applicable uniquement si les données sont prises parmi un petit ensemble connu à l'avance). Si l'on ne précise rien, on entend habituellement par algorithme de tri un algorithme général de tri par comparaison.

Les principales caractéristiques qui permettent de différencier les algorithmes de tri sont : la complexité temporelle, la complexité spatiale, et le caractère stable.

2.1 Complexité

On a vu que pour certains des algorithmes de tri les plus simples la complexité est $O(n^2)$, pour les tris plus élaborés, $O(n \log(n))$.

On peut montrer que la complexité temporelle en moyenne et dans le pire des cas d'un algorithme basé sur une fonction de comparaison ne peut pas être meilleure que $O(n \log(n))$. Les tris qui ne demandent que $n \log(n)$ comparaisons en moyenne sont alors dits optimaux.

Pour certains types de données (entiers, chaînes de caractères de taille bornée), il existe cependant des algorithmes plus efficaces au niveau du temps d'exécution, comme le tri comptage ou le tri par base. Ces algorithmes n'utilisent pas la comparaison entre éléments (la borne $n \log(n)$ ne s'applique donc pas pour eux) mais nécessitent des hypothèses sur les objets à trier. Par exemple, le tri comptage et le tri

par base s'appliquent à des entiers que l'on sait appartenir à l'ensemble $\llbracket 1, m \rrbracket$ avec comme hypothèse supplémentaire pour le tri par base que m soit une puissance de 2 (c'est-à-dire de la forme $m = 2^k$).

2.2 Caractère en place

Un algorithme est dit en place s'il n'utilise qu'un nombre très limité de variables et qu'il modifie directement la structure qu'il est en train de trier. Ceci nécessite l'utilisation d'une structure de donnée adaptée (un tableau par exemple). Ce caractère peut être très important si on ne dispose pas d'une grande quantité de mémoire utilisable.

Remarquons toutefois qu'en général, on ne trie pas directement les données elles-mêmes, mais seulement des références (ou pointeurs) sur ces dernières.

2.3 Caractère stable

Un algorithme est dit stable s'il garde l'ordre relatif des quantités égales pour la relation d'ordre.

Exemple, si on considère la suite d'éléments suivante : (4,1) (3,1) (3,7) (5,6)

que l'on trie par rapport à leur première coordonnée (la clé), deux cas sont possibles, quand l'ordre relatif est respecté et quand il ne l'est pas :

(3,1) (3,7) (4,1) (5,6) (ordre relatif maintenu)

(3,7) (3,1) (4,1) (5,6) (ordre relatif changé)

Lorsque deux éléments sont égaux pour la relation d'ordre (c'est-à-dire qu'ils ont la même clé), l'algorithme de tri conserve l'ordre dans lequel ces deux éléments se trouvaient avant son exécution. Les algorithmes de tri instables peuvent être retravaillés spécifiquement afin de les rendre stables, cependant cela peut être aux dépens de la rapidité et/ou peut nécessiter un espace mémoire supplémentaire.

3 Exercices

Exercice n° 1

Rechercher la complexité spatiale et temporelle des algorithmes classiques, dans le meilleur des cas, dans le pire des cas et dans le cas moyen.

Quels sont les algorithmes stables et/ou en place :

1) Le tri à bulles

```
Algorithme triBulle(t : tableau)
variable
  n, i, j : entier
début
  n ← longueur(t)
  pour i de n-1 à 1 faire
    pour j de 0 à i-1 faire
      si t[j+1] < t[j] alors échanger t[j] et t[j+1] fsi
    fpr
  fpr
```

2) Le tri par sélection

```
Algorithme triSelection(t : tableau)
variable
  n, i, j, min : entier
début
  n ← longueur(t)
  pour i de 0 à n-2 faire
    min ← i
    pour j de i+1 à n-1 faire
      si t[j] < t[min] alors min ← j fsi
    fpr
    si min ≠ i alors échanger t[i] et t[min] fsi
  fpr
fin
```

Ce tri est donc intéressant lorsque les éléments sont aisément comparables, mais coûteux à déplacer dans la structure. Pourquoi ?

3) Le tri insertion

```
Algorithme triInsertion(t : tableau)
variable
  n, i, j, tmp : entier
début
  n ← longueur(t)
  pour i de 1 à n-1 faire
    tmp ← t[i]          // mémorise la valeur à insérer
    j ← i
    tq j>0 et t[j-1]>tmp faire
      t[j] ← t[j-1]    // on décale les valeurs plus grandes
      j ← j-1
    ftq
    t[j] ← tmp
  fpr
fin
```

4) Le tri fusion (ou tri dichotomique)

L'algorithme est naturellement décrit de façon récursive :

- a) Si le tableau n'a qu'un élément, il est déjà trié.
- b) Sinon, séparer le tableau en deux parties à peu près égales.
- c) Trier récursivement les deux parties avec l'algorithme du tri fusion.
- d) Fusionner les deux tableaux triés en un seul tableau trié.

(Java utilise une variante du tri fusion pour ses tris de l'objet Collections)

5) Le tri rapide

La méthode consiste à placer un élément du tableau (appelé pivot) à sa place définitive, en permutant tous les éléments de telle sorte que tous ceux qui sont inférieurs au pivot soient à sa gauche et que tous ceux qui sont supérieurs au pivot soient à sa droite.

Cette opération s'appelle le partitionnement. Pour chacun des sous-tableaux, on définit un nouveau pivot et on répète l'opération de partitionnement. Ce processus est répété récursivement, jusqu'à ce que l'ensemble des éléments soit trié.

Exercice n° 2 Tri par tas

1964

L'idée qui sous-tend cet algorithme consiste à voir le tableau comme un arbre binaire.

- Le premier élément est la racine, le deuxième et le troisième sont les deux descendants du premier élément.
- Ainsi le $n^{\text{ème}}$ élément a pour enfants les éléments $2n + 1$ et $2n + 2$

Si l'indexation se fait à partir de 0 (ce qu'on va supposer ici).

Si le tableau n'est pas de taille $(2^n - 1)$, les branches ne se finissent pas toutes à la même profondeur. Les nœuds de l'arbre sont placés dans le tableau ligne par ligne (1 valeur pour la 1^{ère} ligne, 2 valeurs pour la 2^{ème}, ..., 2^{k-1} valeurs pour la $k^{\text{ème}}$), chaque ligne étant décrite de gauche à droite.

Dans l'algorithme, on cherche à obtenir un tas, c'est-à-dire un arbre binaire vérifiant les propriétés suivantes :

- la différence maximale de profondeur entre deux feuilles est de 1 (càd toutes les feuilles se trouvent sur la dernière ou sur l'avant-dernière ligne) ;
- les feuilles de profondeur maximale sont "tassées" sur la gauche.
- chaque nœud est de valeur supérieure à celles de ses deux fils, pour un tri ascendant.

Le tableau est constitué de deux parties :

- le tas (au départ, tout le tableau),
- le tableau trié (à la fin, tout le tableau).

L'opération de base de ce tri est le tamisage, qui consiste à échanger un nœud avec le plus grand de ses fils, et de refaire l'opération de tamisage sur le fils échangé.

Donc le tri consiste à :

- Construire un tas à partir d'un arbre quelconque, pour cela, on tamise les nœuds, de droite à gauche et de bas en haut.

Donc on commence au milieu du tableau en remontant vers le début (car la seconde moitié du tableau ne contient que des feuilles).

- Il en découle que la racine du tas (le premier élément) contient la valeur maximale de l'arbre. Donc on échange la racine avec le dernier élément du tableau. Et maintenant, on considère le tableau moins son dernier élément.
- On applique le tamisage à partir du premier élément du tableau, donc on revient au point précédent.

- 1) Tester cet algorithme sur $\text{tab} = \{10, 4, 8, 5, 12, 2, 6, 11, 3, 9, 7, 1\}$
- 2) Quelle est la complexité temporelle ?
- 3) Quelle est la complexité spatiale ?
- 4) Expliquer pourquoi c'est un tri non stable mais en place.

Exercice n° 3 Tri comptage

1954

Le tri comptage est un algorithme de tri non comparatif. Il trie un tableau en comptant les occurrences de chaque élément unique. Il effectue un hachage partiel du comptage des éléments uniques et effectue ensuite des calculs pour trouver l'indice de chaque élément dans le tableau final trié. C'est un algorithme assez rapide mais qui ne convient pas aux grands ensembles de données. Il est utilisé comme sous-programme dans le tri par base.

Supposons que nous ayons un tableau non trié `tab` contenant n éléments.

- Trouver l'élément maximum `max` et l'élément minimum `min` dans le tableau.
 - Initialiser un tableau `comptage` de longueur `max-min+1` avec tous les éléments mis à 0.
 - Initialiser un tableau `resultat` de même taille que le tableau d'entrée `tab`.
 - Enregistrer le nombre de tous les éléments dans le tableau `comptage` en soustrayant `min` et en utilisant la différence comme index.
 - Cumuler la somme des éléments dans le tableau `comptage`. Le tableau `comptage` contient maintenant la position de chaque élément dans le tableau trié.
 - En partant de la fin, prendre les éléments de `tab` et faire ce qui suit :
 - Calculer l'index des éléments sous la forme `comptage[tab[i]-min]-1` et stockez `tab[i]` dans `resultat`.
 - Diminuer `comptage[tab[i]-min]` de 1.
 - Stocker les éléments `resultat` dans le tableau original `tab`.
- 1) Tester cet algorithme sur `tab = {(18,A), (17,B), (18,C), (19,D), (17,E), (18,F), (16,G)}`
 - 2) Quelle est la complexité temporelle ?
C'est la même chose que la complexité temporelle dans le pire des cas.
 - 3) Quelle est la complexité spatiale ?
 - 4) Expliquer pourquoi c'est un tri stable mais pas en place.

Exercice n° 4 Tri par base

1887

Le tri par base, ou tri radix, est un algorithme de tri, utilisé pour ordonner des éléments identifiés par une clef unique. Chaque clef est une chaîne de caractères ou un nombre que le tri par base trie selon l'ordre lexicographique. Cet algorithme a besoin d'être couplé avec un ou plusieurs algorithmes de tri stable. Si chaque chiffre a un nombre limité de valeur, le tri comptage est souvent utilisé.

Le principe de l'algorithme est le suivant :

- On considère le chiffre le moins significatif de chaque clef.
 - On trie la liste des éléments selon ce chiffre avec un algorithme de tri stable. Il y a une conservation de l'ordre des éléments ayant le même chiffre car le tri est stable.
 - On répète l'étape 2 en regardant le chiffre suivant.
- 1) Tester cet algorithme sur `tab = {170, 45, 75, 90, 2, 24, 802, 66, 42, 77, 21}`. Ici, le chiffre le moins significatif est le chiffre des unités, ensuite celui des dizaines et enfin celui des centaines.
 - 2) Quelle est la complexité temporelle ?
 - 3) Quelle est la complexité spatiale ?
 - 4) Expliquer pourquoi c'est un tri stable mais pas en place.